

Mise en place d'un écosystème d'IA
420-A57-BB

TRAVAIL PRATIQUE

Module 2 : Mettre en place un environnement matériel dans le contexte de l'IA

Durée estimée	2 heures
Prérequis	Module 2 (Cerner les besoins matériels dans le contexte de l'IA)
Environnement	Ubuntu 22.04 LTS (natif ou VM) OU Windows 10/11 + WSL2

Préparation : identifier votre environnement

Ce TP est conçu pour fonctionner sur deux environnements. Identifiez le vôtre avant de commencer.

Environnement A : Ubuntu natif	Environnement B : Windows + WSL2
Vous êtes sur une machine (physique ou VM) avec Ubuntu installé directement. Vous avez un bureau Ubuntu ou un terminal Ubuntu.	Vous êtes sur Windows 10/11 et vous accédez à Linux via WSL2 (Windows Subsystem for Linux). Vous ouvrez un terminal Ubuntu depuis Windows.
Ouvrez un terminal Ubuntu directement.	Ouvrez le menu Démarrer → tapez 'Ubuntu' → lancez le terminal WSL2.

Convention dans ce TP : les commandes identiques sur les deux environnements sont affichées en gris. Quand les commandes diffèrent, vous verrez deux blocs distincts : Ubuntu natif (fond bleu clair) et Windows + WSL2 (fond violet clair). Suivez uniquement le bloc correspondant à votre environnement.

Si vous êtes sur Windows : vérifier que WSL2 est prêt

Si WSL2 n'est pas encore installé, ouvrez PowerShell en administrateur et exécutez :

Windows + WSL2

```
# Dans PowerShell (Windows) :
wsl --install -d Ubuntu -2
# Redémarrez Windows, puis lancez Ubuntu depuis le menu Démarrer
# Créez votre utilisateur et mot de passe Linux
```

Puis vérifiez que WSL2 fonctionne :

Windows + WSL2

```
wsl --list --verbose
# Doit afficher : Ubuntu-22.04 Running 2
```

Important WSL2 : une fois dans le terminal Ubuntu WSL2, toutes les commandes Linux fonctionnent normalement. La seule différence majeure concerne les pilotes GPU (Partie 2) : sous WSL2, les drivers NVIDIA sont gérés côté Windows, pas côté Linux.

Objectifs du TP

Ce TP vous amènera à mettre en place un environnement complet pour l'IA. À la fin, vous aurez installé et validé un système prêt pour l'entraînement de modèles.

Consigne : Pour chaque étape, fournissez une capture d'écran de la commande ET de son résultat. Les étapes sans preuve ne seront pas notées.

Partie 1 — Système d'exploitation (20 points)

Exercice 1.1 — Identification du système (5 pts)

Ouvrez un terminal (Ubuntu natif ou WSL2) et exécutez :

```
a) lsb_release -a
b) uname -r
c) hostnamectl
```

Question : Quelle version d'Ubuntu utilisez-vous ? Est-ce une version LTS ? Expliquez ce que signifie LTS et pourquoi c'est important pour l'IA. Si vous êtes sous WSL2, précisez-le dans votre réponse.

Exercice 1.2 — Mises à jour et outils (10 pts)

a) Mise à jour du système (3 pts)

```
sudo apt update
sudo apt upgrade -y
```

Question : Quelle est la différence entre apt update et apt upgrade ?

b) Installation d'outils essentiels (4 pts)

```
sudo apt install -y build-essential curl wget git htop net-tools

# Vérification :
git --version
curl --version | head -1
```

c) Espace disque (3 pts)

```
df -h
lsblk
```

Note WSL2 : sous WSL2, le disque affiché est un disque virtuel (.vhdx). L'espace disponible dépend de votre disque Windows. C'est normal de voir des résultats différents d'Ubuntu natif.

Question : Identifiez l'espace disponible. Est-ce suffisant pour stocker un dataset de 50 Go ?

Exercice 1.3 — Questions théoriques (5 pts)

Q1 (2 pts) : Pour un serveur d'entraînement Deep Learning 24h/24, quel OS choisir (Ubuntu Desktop, Ubuntu Server, Windows 11 Pro) ? Justifiez avec 3 raisons.

Q2 (1.5 pts) : Qu'est-ce que WSL2 ? Comment fonctionne-t-il ? Quels sont ses avantages et ses limites par rapport à Ubuntu natif pour un projet d'IA ?

Q3 (1.5 pts) : Pourquoi séparer la partition système (/) de la partition données (/home) ? Donnez un scénario concret. Cette séparation est-elle possible sous WSL2 ?

Partie 2 — Pilotes et accélération GPU (25 points)

Important : la gestion des pilotes GPU diffère fortement selon votre environnement. Suivez bien les instructions correspondant au vôtre.

Exercice 2.1 — Détection du matériel (5 pts)

Ces commandes fonctionnent sur les deux environnements :

```
a) lspci | grep -i nvidia      # (peut ne rien afficher sous WSL2)
b) lspci | grep -i nvidia
c) cat /proc/cpuinfo | grep 'model name' | head -1
d) free -h
```

Note WSL2 : lspci peut ne pas afficher le GPU sous WSL2, c'est normal. Le GPU est géré par Windows. Passez à l'étape nvidia-smi pour vérifier.

Question : Quel matériel est détecté ? Si aucun GPU NVIDIA, expliquez pourquoi.

Exercice 2.2 — Installation et vérification des pilotes (10 pts)

Environnement A — Ubuntu natif :

Ubuntu natif

```
# a) Vérifier les pilotes recommandés (2 pts)
ubuntu-drivers devices

# b) Installer le pilote (3 pts)
sudo apt install -y nvidia-driver-535
# OU : sudo ubuntu-drivers autoinstall
sudo reboot      # Redémarrage OBLIGATOIRE

# c) Vérification après redémarrage (3 pts)
nvidia-smi

# d) CUDA Toolkit (2 pts)
sudo apt install -y nvidia-cuda-toolkit
nvcc --version
```

Environnement B — Windows + WSL2 :

Sous WSL2, les pilotes NVIDIA sont installés côté Windows, pas côté Linux.

Windows + WSL2

```
# a) Installer le driver NVIDIA pour Windows (depuis le site NVIDIA)
#   Téléchargez depuis : https://www.nvidia.com/drivers
#   Installez le driver Windows normalement (pas besoin de driver Linux)

# b) Vérifier que le GPU est visible depuis WSL2 (3 pts)
nvidia-smi      # Doit fonctionner directement si le driver Windows est installé

# c) Installer CUDA Toolkit dans WSL2 (2 pts)
```

```
# ATTENTION : ne PAS installer nvidia-driver dans WSL2 !  
# Installer uniquement le toolkit CUDA :  
sudo apt install -y nvidia-cuda-toolkit  
nvcc --version
```

Pour les deux : annotez votre capture nvidia-smi en identifiant le nom du GPU, la version driver, la VRAM et la version CUDA.

Sans GPU NVIDIA : si vous n'avez pas de GPU (VM sans GPU, carte AMD/Intel), décrivez théoriquement chaque commande et son rôle. Vous obtiendrez 7/10 max.

Exercice 2.3 — Questions théoriques GPU (10 pts)

Q1 (3 pts) : Décrivez la chaîne GPU en 3 couches (Driver → CUDA → cuDNN). Pour chacune : rôle + commande de vérification.

Q2 (3 pts) : Un étudiant obtient 'CUDA not available'. Listez 4 causes possibles dans l'ordre de vérification. Précisez si la cause diffère entre Ubuntu natif et WSL2.

Q3 (2 pts) : Quelle est la différence entre CUDA et cuDNN ? Peut-on utiliser PyTorch sans cuDNN ?

Q4 (2 pts) : Pourquoi sous WSL2, il ne faut PAS installer le driver NVIDIA côté Linux ? Que se passe-t-il si on le fait ?

Partie 3 — Configuration réseau (15 points)

Exercice 3.1 — Diagnostic réseau (8 pts)

a) Informations réseau (3 pts)

```
ip addr show
ip route show
cat /etc/resolv.conf
```

Note WSL2 : sous WSL2, l'adresse IP est celle d'un réseau virtuel interne (172.x.x.x). Le DNS pointe souvent vers l'IP Windows. C'est le comportement attendu.

Question : Identifiez : adresse IP, interface réseau, passerelle, serveur(s) DNS. Si WSL2, expliquez pourquoi l'IP diffère de celle de Windows.

b) Tests de connectivité (3 pts)

```
ping -c 4 8.8.8.8
ping -c 4 google.com
curl -s -o /dev/null -w '%{http_code}' https://pypi.org
```

Question : Si ping 8.8.8.8 fonctionne mais pas google.com, quel problème ? Si curl retourne 200, qu'est-ce que ça signifie pour pip ?

c) Bande passante (2 pts)

```
wget --output-document=/dev/null https://speed.hetzner.de/100MB.bin 2>&1 | tail -2
```

Question : Vitesse obtenue ? Temps estimé pour télécharger un dataset de 50 Go ? (Calculez)

Exercice 3.2 — SSH (7 pts)

a) Installer SSH (3 pts)

```
sudo apt install -y openssh-server
sudo systemctl enable ssh
sudo systemctl start ssh
sudo systemctl status ssh
```

Note WSL2 : systemctl peut ne pas fonctionner sous WSL2. Utilisez plutôt : sudo service ssh start puis sudo service ssh status

b) Test connexion (2 pts)

```
ssh localhost
```

c) Question (2 pts) : Pourquoi SSH est indispensable en IA ? Scénario concret d'utilisation quotidienne par un data scientist.

Partie 4 — Environnement logiciel (30 points)

Les commandes de cette partie sont identiques sur Ubuntu natif et WSL2.

Exercice 4.1 — Python et venv (8 pts)

a) Installation Python (2 pts)

```
sudo apt install -y python3 python3-pip python3-venv
python3 --version
pip3 --version
```

b) Environnement virtuel (3 pts)

```
python3 -m venv ~/env_ia
source ~/env_ia/bin/activate
which python
which pip
```

Question : Pourquoi utiliser un venv ? Donnez un exemple concret de conflit évité.

c) Bibliothèques essentielles (3 pts)

```
pip install numpy pandas matplotlib scikit-learn jupyter

python -c "import numpy; print('NumPy:', numpy.__version__)"
python -c "import pandas; print('Pandas:', pandas.__version__)"
python -c "import sklearn; print('Scikit-learn:', sklearn.__version__)"
```

Exercice 4.2 — Jupyter Notebook (7 pts)

a) Lancement (2 pts)

```
jupyter notebook --no-browser --port=8888
```

WSL2 : copiez l'URL affichée ([http://localhost:8888/...](http://localhost:8888/)) et ouvrez-la dans votre navigateur Windows. Le port forwarding est automatique.

b) Notebook de test (5 pts) :

Cellule 1 — NumPy :

```
import numpy as np
matrice = np.random.randn(3, 3)
print('Matrice :', matrice)
print(f'Moyenne : {matrice.mean():.4f}')
```

Cellule 2 — Matplotlib :

```
import matplotlib.pyplot as plt
x = np.linspace(0, 10, 100)
plt.figure(figsize=(8, 4))
plt.plot(x, np.sin(x), color='teal', linewidth=2)
plt.title('Test - Sin(x)')
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

Exercice 4.3 — Framework ML (10 pts)

Choisissez PyTorch (recommandé) OU TensorFlow.

Installation PyTorch :

```
# Avec GPU NVIDIA (Ubuntu natif OU WSL2 avec driver Windows installé) :
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121

# Sans GPU (CPU uniquement) :
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

Installation TensorFlow :

```
pip install tensorflow
```

Vérification (5 pts) :

```
import torch, sys
print(f'Python : {sys.version}')
print(f'PyTorch : {torch.__version__}')
print(f'CUDA disponible : {torch.cuda.is_available()}')
if torch.cuda.is_available():
    print(f'GPU : {torch.cuda.get_device_name(0)}')
x = torch.randn(1000, 1000)
y = torch.matmul(x, x)
print(f'Calcul réussi ! Taille : {y.shape}')
```

Mini-projet (5 pts) — Classification Iris :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print(f'Précision : {accuracy_score(y_test, predictions):.2%}')
print(classification_report(y_test, predictions, target_names=iris.target_names))
```

Exercice 4.4 — Questions théoriques (5 pts)

Q1 (2 pts) : Comparez PyTorch et Keras. Lequel recommander à un débutant ?

Q2 (1.5 pts) : Qu'est-ce qu'un IDE ? Citez 3 IDE pour l'IA. Avantage de Jupyter vs VS Code pour l'exploration de données ?

Q3 (1.5 pts) : Pourquoi Python domine en IA ? 3 raisons. Quel langage pour améliorer les performances ?

Partie 5 — Validation finale (10 points)

Exercice 5.1 — Script de validation (5 pts)

Écrivez un script Python (`validation_env.py`) qui vérifie automatiquement : versions Python/pip, bibliothèques (NumPy, Pandas, Matplotlib, Scikit-learn), framework ML, disponibilité GPU, et indique si vous êtes sur Ubuntu natif ou WSL2. Affichage avec indicateurs OK/ERREUR.

Exercice 5.2 — Rapport de synthèse (5 pts)

Rédigez un rapport (1 page max) : configuration machine, environnement utilisé (Ubuntu natif ou WSL2), logiciels installés, problèmes rencontrés et solutions, auto-évaluation de la machine pour le Deep Learning.

Barème récapitulatif

Partie	Points	%
Partie 1 — Système d'exploitation	20	20%
Partie 2 — Pilotes et GPU	25	25%
Partie 3 — Configuration réseau	15	15%
Partie 4 — Environnement logiciel	30	30%
Partie 5 — Validation et rapport	10	10%
TOTAL	100	100%

Remise : PDF avec captures + notebooks (.ipynb) + script `validation_env.py` → archive ZIP : `NOM_Prenom_TP2.zip`